

Module	Topics Covered
1. Introduction	- What is DSA?- Time & Space Complexity- Linear vs Non-Linear Data Structures
2. Searching	- Linear Search- Binary Search (iterative & recursive)- First/Last occurrence problems
3. Arrays	- Basics: traversal, insertion, deletion, update- Problems: reverse array, find max/min, second largest element- Practice coding exercises
4. Sorting (Basic)	- Bubble Sort- Selection Sort- Insertion Sort- Comparison of complexities- Hands-on problems
5. Sorting (Advanced)	- Merge Sort (divide & conquer, recursive implementation)- Quick Sort (partition logic, best/worst case)- Applications of sorting- Problems: sort student records, k-th smallest element
6. Stack	- Concept (LIFO)- Implementation using list & linked list- Applications: balanced parentheses, undo operation- Hands-on problems
7. Queue	- Concept (FIFO)- Simple Queue implementation- Circular Queue implementation- Applications: job scheduling, printer queue- Practice coding
8. Linked List (Part 1)	- Node & pointer structure- Singly Linked List: creation, insertion, deletion, traversal- Problems: reverse a linked list, find middle node
9. Linked List (Part 2)	- Doubly Linked List implementation- Circular Linked List implementation- Detecting loops in Linked List- Practice problems
10. Trees (Part 1)	- Tree concepts & terminology- Binary Tree basics- Tree Traversals: Inorder, Preorder, Postorder- Hands-on coding
11. Trees (Part 2)	- Binary Search Tree (BST): insertion, deletion, search- Applications of BST- Problems: height of tree, lowest common ancestor
12. Graphs	- Graph basics & terminology- Representation: Adjacency List & Matrix- BFS traversal- DFS traversal- Applications: maze solving, shortest path basics

Data Structure & Algorithm

Introduction

1. What is Data?

Definition: Data is a raw, unorganized collection of facts and figures.

It can be numbers, characters, symbols, or anything that represents a value but has no inherent meaning by itself.

Examples:

- 12345
- "ABC"
- Sensor readings like 98.6, 101.2, 97.8

Think of **data** as the **raw ingredients** in your kitchen. On their own, they don't mean much.

2. What is Information?

Definition: Information is processed, organized, or structured data that has meaning and can be used for decision-making.

It is essentially **data + context**.

Examples:

- Data: 12345 → Information: *"This is the roll number of a student."*
- Data: 98.6, 101.2, 97.8 → Information: *"These are temperature readings of patients."*
- Data: "ABC" → Information: *"This is the name of a company."*

So, **information = useful meal cooked from raw ingredients (data)**.

3. What is a Data Structure?

Definition: A data structure is a way of organizing, storing, and managing data in a computer so that it can be used efficiently.

It provides a systematic method to access and modify data.

Examples:

- Arrays → store elements sequentially
- Linked List → connect elements using pointers
- Stack → LIFO (Last In, First Out)
- Queue → FIFO (First In, First Out)
- Tree → hierarchical structure
- Graph → nodes and edges

Think of a **data structure as the container or arrangement you put your ingredients (data) in**, like a fridge, jar, or shelf that makes accessing them easier.

4. What is an Algorithm?

An **algorithm** is a step-by-step procedure or set of rules to solve a specific problem.

It is independent of programming language.

For the same problem, multiple algorithms may exist with different performance characteristics.

Examples of Algorithms:

- Searching algorithms (Linear Search, Binary Search)
 - Sorting algorithms (Bubble Sort, Merge Sort, Quick Sort)
 - Graph algorithms (BFS, DFS, Dijkstra's)
-

5. Why do we need Data Structures?

1. **Efficient Data Access:** Instead of searching randomly, data structures allow quick access (e.g., Binary Search on sorted arrays).
2. **Memory Management:** They help in organizing memory (e.g., linked lists dynamically allocate space).
3. **Faster Operations:** Certain structures speed up insertion, deletion, and search (e.g., hash tables, BST).
4. **Problem Solving:** Many real-world problems (networks, compilers, operating systems, databases) are solved using appropriate data structures.
5. **Scalability:** Large applications (like Google Search, social media, or Amazon's cart system) rely heavily on choosing the right data structures.

Time Complexity

What is Time Complexity?

- **Definition:** Time complexity is the measure of how the running time of an algorithm grows with the size of the input.
- It helps us analyze the **efficiency** of an algorithm without actually coding it on every possible machine.
- We measure it in terms of **input size (n)**.

Example: If you loop through an array of size n once, it takes roughly n steps $\rightarrow$ Time complexity = **$O(n)$** .

Types of Time Complexity

When analyzing algorithms, we usually look at three cases:

1. Best Case (θ)

- The minimum time an algorithm will take for an input.
- Example: Searching for the first element in a linear search.
- Not very reliable for practical analysis because real-world data rarely matches best case.

2. Average Case (Ω)

- The expected time over all possible inputs.
- Example: In linear search, if the element is somewhere in the middle, we check half the array on average.

3. Worst Case (O)

- The maximum time an algorithm will take.
 - Example: In linear search, when the element is not present at all, we check the entire array.
 - **Most commonly used in analysis** because it guarantees performance under any condition.
-

Commonly Used Big-O Notations

Big-O notation is used to describe time complexity in a machine-independent way. Here are the most common ones:

- **$O(1)$:** `arr[5]` $\rightarrow$ directly accessing an element.
- **$O(n)$:** Looping through list once.
- **$O(n^2)$:** Looping through list inside another loop.
- **$O(\log n)$:** Repeatedly dividing search space in half (Binary Search).
- **$O(n \log n)$:** Sorting using efficient algorithms.

Notation	Name	Example
O(1)	Constant Time	Accessing an element in an array by index
O(log n)	Logarithmic Time	Binary Search
O(n)	Linear Time	Linear Search, simple loops
O(n log n)	Linearithmic Time	Merge Sort, Quick Sort (average case)
O(n²)	Quadratic Time	Nested loops, Bubble Sort, Insertion Sort, Selection Sort
O(n³)	Cubic Time	Triple nested loops
O(2ⁿ)	Exponential Time	Recursive Fibonacci (naive)
O(n!)	Factorial Time	Solving Traveling Salesman by brute force

So, time complexity is like predicting **how long your attack combo will take depending on the number of enemies (n)**. Some moves (algorithms) scale better than others.

1. $O(1)$ – Constant Time

The algorithm takes the same amount of time regardless of the size of the input. Execution does not depend on n .

Example: Accessing an element in an array by index

```
arr = [10, 20, 30, 40, 50]
print(arr[2]) # Accessing the 3rd element
```

Whether the array has 5 elements or 5 million, accessing `arr[2]` takes the same time. That's why this is $O(1)$.

2. $O(n)$ – Linear Time

The algorithm's time increases linearly with the size of input n . For every element, you perform some operation.

Example: Linear Search

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i # Found at index i
    return -1 # Not found
```

```
arr = [5, 8, 12, 20, 15]
print(linear_search(arr, 20)) # Output: 3
```

If the array has 5 elements, at worst we check 5 times. If the array has 1,000 elements, at worst we check 1,000 times. So the time taken grows **proportionally** with n .

3. $O(\log n)$ – Logarithmic Time

The algorithm reduces the problem size by half (or a constant factor) each step. Execution grows slowly compared to input size.

Example: Check if a number is a palindrome (by dividing digits)

```
def is_palindrome(num):
    original = num
    rev = 0
    while num > 0:
        digit = num % 10
        rev = rev * 10 + digit
        num //= 10 # reducing number by factor of 10 each time
    return original == rev

print(is_palindrome(12321)) # True
print(is_palindrome(12345)) # False
```

- Each loop removes one digit (divide by 10).
 - If the number has **d digits**, we run the loop **d times**.
 - But $d = \log_{10}(n)$.
 - So time complexity = **$O(\log n)$** .
-

4. $O(n \log n)$ – Linear - Logarithmic Time

An algorithm that does n operations, and each operation involves a $\log n$ step. Common in **sorting** and divide & conquer.

Example for $O(n \log n)$: Print all factors of numbers from 1 to n

```
def factors(n):
    for i in range(1, n+1):
        print(f"{i}: ", end="")
        j = 1
        while j * j <= i: # checking divisors up to  $\sqrt{i}$ 
            if i % j == 0:
                print(j, end=" ")
                if j != i // j:
                    print(i // j, end=" ")
            j += 1
        print()

factors(10)
```

Why $O(n \log n)$?

- For each number i from 1 to $n \rightarrow$ **n loop**.
- Inside, we check divisors up to $\sqrt{i} \rightarrow$ about **$\log i$ steps**.
- Overall complexity = **$O(n \log n)$** (technically closer to $n\sqrt{n}$, but using divisor pair trick it reduces).

So, **$O(n \log n)$** usually means:

- You handle each element (n)
 - And you also divide/combine things ($\log n$)
-

5. $O(n^2)$ – Quadratic Time

The algorithm's time grows proportional to the square of input size. Typically comes from **nested loops**.

Example: Printing all pairs in a list

```
arr = [1, 2, 3, 4]
```

```
for i in arr:
    for j in arr:
        print(i, j)
```

- Outer loop runs n times.
- Inner loop also runs n times for each outer loop.
- Total = $n \times n = n^2$.

If $n = 10$, operations = 100. If $n = 1000$, operations = 1,000,000. So time grows much faster than linear.

6. $O(2^n)$ – Exponential Time

- **Definition:** The algorithm doubles in time complexity for every additional input size.
- Common in recursive problems with many branching paths.

Example: Fibonacci with recursion (naive)

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

print(fib(5)) # Output: 5
```

- For $\text{fib}(n)$, you call $\text{fib}(n-1)$ and $\text{fib}(n-2)$.
- Each of those calls spawns two more calls.
- This branching forms a **binary tree of calls** with about 2^n nodes.

If $n = 10$, calls ~ 1024 . If $n = 50$, calls explode to **over 1 trillion**.

That's why exponential time is impractical for large inputs.

7. $O(n!)$ – Factorial Time

We'll generate all permutations of a string **without any module**, just recursion:

```
def permutations(s, chosen=""):
    if not s: # nothing left to choose
        print(chosen)
        return
    for i in range(len(s)):
        # pick one character
        c = s[i]
        # remaining string after removing c
        remaining = s[:i] + s[i+1:]
        # recursive call
        permutations(remaining, chosen + c)

# Example
permutations("abc")
```

Output:

```
abc
acb
bac
bca
cab
cba
```

Explanation:

- At each step, we “choose” one character, then recursively arrange the remaining characters.
 - If string length is n , recursion branches into $n!$ possibilities.
 - "abc" → 6 permutations.
 - "abcd" → 24 permutations.
 - "abcdef" → 720 permutations.
 - Growth: $n!$ factorial.
-

Space Complexity

1. What is Space Complexity?

- **Definition:** Space complexity measures the **amount of memory** an algorithm uses relative to input size n .
 - Includes:
 - Memory for input
 - Memory for variables and data structures
 - Memory used by function calls (stack for recursion)
 - Helps determine **how memory-efficient an algorithm is**, just like time complexity tells us about speed.
-

Types of Space Complexity

a) Constant Space – $O(1)$

Uses **fixed amount of memory** regardless of input size.

- Example: Swapping two numbers

```
a = 5
b = 10
a, b = b, a
print(a, b)
```

Only two variables used → memory doesn't grow with n .

b) Linear Space – $O(n)$

Memory grows **proportionally to input size**.

- Example: Storing elements of a list

```
n = 5
arr = [i for i in range(n)] # list of n elements
print(arr)
```

If $n = 5$, list has 5 elements; if $n = 1000$, list has 1000 elements.

Memory grows linearly with n .

c) Auxiliary Space

Memory **used by the algorithm itself**, excluding input. Often arises in **recursion or temporary variables**.

- Example: Recursive factorial

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n-1)  
  
print(factorial(5))
```

- Input n itself is not counted.
- Recursion uses **stack memory** → auxiliary space = $O(n)$.
- Iterative factorial would only use $O(1)$ auxiliary space.

Type	Description	Example	Space Complexity
Constant	Fixed memory, independent of input	Swapping two numbers	$O(1)$
Linear	Memory grows with input	Storing a list of n elements	$O(n)$
Auxiliary	Extra memory used by algorithm	Recursive factorial	$O(n)$

Searching

What is Searching?

- **Definition:** Searching is the process of finding the **location (index or position) of an element** in a data structure.
- In programming, we often search for a number, a string, or any element in arrays, lists, or other structures.
- **Goal:** Quickly locate the desired element without checking every single item if possible.

Example scenario:

- You have a list of student roll numbers and want to find the roll number 105. Searching is the process of finding where 105 is in that list.
-

Types of Searching

- 1) **Linear Search (Sequential Search)**
 - 2) **Binary Search**
-

1. Linear Search

Definition: Check each element one by one until the desired element is found or the list ends.

Characteristics:

1. Works on **unsorted or sorted data**.
2. Simple, but **slow for large data** because it may check every element.

Algorithm Idea:

- Start from the **first element** of the list.
 - Compare it with the target value.
 - If found → return the position.
 - If not → move to the next element.
 - Continue until the element is found or the list ends.
-

Python Implementation

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i # return index if found
    return -1 # return -1 if not found

# Example
numbers = [10, 25, 47, 3, 89, 25]
print(linear_search(numbers, 47)) # Output: 2
print(linear_search(numbers, 100)) # Output: -1
```

Time Complexity

- $O(n)$ → If the element is at the **last position** or **not present**.
-

Space Complexity

- Uses only a few variables (index, target check).
 - No extra memory used → **$O(1)$** .
-
- Linear Search is simple but slow for large datasets.
 - It's the go-to method when the list is **unsorted**.
-

Binary Search

Definition: Search in a **sorted data structure** by repeatedly dividing the search space in half.

Characteristics:

- 1) Requires **sorted data**.
- 2) Much faster than linear search for large data.

Concept:

- Works only on **sorted data**.
 - Instead of checking elements one by one, it uses **divide & conquer**:
 1. Compare the target with the **middle element**.
 2. If `target == middle` → found.
 3. If `target < middle` → search in the **left half**.
 4. If `target > middle` → search in the **right half**.
 - Repeat until element is found or search space is empty.
-

5. Python Implementation (Iterative)

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2 # middle index

        if arr[mid] == target:
            return mid # element found
        elif arr[mid] < target:
            left = mid + 1 # search in right half
        else:
            right = mid - 1 # search in left half

    return -1 # not found

# Example
numbers = [3, 10, 25, 47, 56, 89]
print(binary_search(numbers, 47)) # Output: 3
print(binary_search(numbers, 100)) # Output: -1
```

Time Complexity

Worst Case: $O(\log n)$ → search space keeps halving until exhausted.

Space Complexity

- Iterative version $\rightarrow O(1)$.
 - Recursive version (because of stack calls) $\rightarrow O(\log n)$.
-

2. Binary Search (Recursive)

Python Implementation

```
def binary_search_recursive(arr, target, left, right):
    # Base condition: element not found
    if left > right:
        return -1

    mid = (left + right) // 2
    if arr[mid] == target:
        return mid # element found
    elif arr[mid] < target:
        # search in right half
        return binary_search_recursive(arr, target, mid + 1, right)
    else:
        # search in left half
        return binary_search_recursive(arr, target, left, mid - 1)

# Example
numbers = [3, 10, 25, 47, 56, 89]
print(binary_search_recursive(numbers, 47, 0, len(numbers)-1)) # Output: 3
print(binary_search_recursive(numbers, 100, 0, len(numbers)-1)) # Output: -1
```

Explanation

- Function calls itself with **reduced search space**.
- Each call reduces the problem size by half.
- If target is found → return its index.
- If not found → eventually left > right, return -1.

Time & Space Complexity

- **Time Complexity:** $O(\log n)$ (since we divide in half each step).
- **Space Complexity:** $O(\log n)$ (because recursive calls use stack memory).

Iterative version is more **space-efficient** ($O(1)$),
while recursive is more **elegant & easy to understand**.

Linear vs Non-Linear Data Structures

1. Linear Data Structures

- **Definition:** Data elements are arranged in a **sequential, linear order**.
Each element has a **unique predecessor and successor** (except the first and last elements).

Key Idea: You can traverse all elements one by one in a single run.

Examples:

1. **Array** – Elements stored one after another.
2. **Linked List** – Nodes connected sequentially using pointers.
3. **Stack** – Last In, First Out (LIFO).
4. **Queue** – First In, First Out (FIFO).

Characteristics:

- Easy to implement.
 - Simple traversal.
 - Limited flexibility for hierarchical relationships.
-

2. Non-Linear Data Structures

- **Definition:** Data elements are **not stored sequentially**.
- Elements can have **multiple relationships** (like parent-child or graph connections).
- **Memory storage** may be scattered.
- **Key Idea:** Traversal may require **more complex techniques** like recursion or graph traversal.

Examples:

1. **Tree** – Hierarchical structure with a root and child nodes.
 - Binary Tree, Binary Search Tree, AVL Tree, etc.
2. **Graph** – Nodes (vertices) connected with edges.
 - Directed/Undirected, Weighted/Unweighted, etc.

Characteristics:

- Can represent complex relationships.
 - Traversal can be more challenging (DFS, BFS).
 - Useful for real-world structures like networks, hierarchies, and maps.
-

Feature	Linear	Non-Linear
Data order	Sequential	Hierarchical / Graph-based
Traversal	Single run	Requires recursion or complex traversal
Memory	Contiguous or sequential	Can be scattered
Examples	Array, Linked List, Stack, Queue	Tree, Graph
Use-case	Simple relationships	Complex relationships (networks, hierarchy)

Array

What is an Array?

- **In DSA** → An **array** is a **collection of elements of the same data type** stored in **contiguous (continuous) memory locations**.

- Example: [10, 20, 30, 40]
- Here, all elements are integers, and they sit side-by-side in memory.
- Access is **O(1)** because we can directly calculate the memory address:

$$\text{Address}(i) = \text{Base_Address} + i \times (\text{size_of_element})$$

- **In Python** → We don't have "true C-style arrays." Instead:
 - **List** = dynamic, can store mixed data types, internally pointers (not continuous).
 - **NumPy Array** (numpy.ndarray) = **the real deal** when it comes to DSA/ML work. Continuous memory, same-type elements, optimized with C under the hood.

◆ NumPy Array Basics

```
import numpy as np

# Creating a NumPy array
arr = np.array([10, 20, 30, 40])

print("Array:", arr)
print("Type:", type(arr))
print("Data Type:", arr.dtype)
print("Shape:", arr.shape)
```

Output:

```
Array: [10 20 30 40]
Type: <class 'numpy.ndarray'>
Data Type: int32 (or int64 depending on system)
Shape: (4,)
So NumPy array is homogeneous + continuous, like real arrays.
```

Difference: List vs NumPy Array

Feature	Python List	NumPy Array
Data Type	Can store mixed types	Must be same type
Memory	Discontinuous (pointers)	Continuous block
Speed	Slower (Python overhead)	Faster (C-optimized)
Operations	Element by element (loops)	Vectorized (whole array at once)

Example	[1, "hi", 3.5]	np.array([1,2,3])
---------	----------------	-------------------

bool → int → float → complex → string (Unicode) → Object

Indexing in NumPy

Yes! Indexing works like lists, but faster.

```
print(arr[0]) # 10
print(arr[-1]) # 40 (last element)
print(arr[1:3]) # [20 30] → slicing
print(arr[:,2]) # [10 30] → step slicing
```

Slicing Explanation (How it Works?)

- Slicing doesn't copy values immediately, it creates a **view** (reference) of the original array in NumPy (unless forced to copy).
- This is efficient because NumPy just calculates an **offset in memory** and jumps directly.

Example:

```
sub_arr = arr[1:3]
sub_arr[0] = 999

print("Original:", arr) # [ 10 999 30 40 ]
```

Changes reflect in the original array because both share the same memory (unless you use `.copy()`).

So in short:

- Array in **DSA** → contiguous, same type.
 - NumPy Array in **Python** → true representation of DSA-style arrays.
 - Indexing/Slicing is possible because elements sit continuously, so NumPy just calculates offsets.
-

2D Arrays (Matrix)

Think of it as a **list of lists**, but in NumPy it's stored as one continuous block of memory.

```
import numpy as np

arr2d = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

print("2D Array:\n", arr2d)
print("Shape:", arr2d.shape) # (3, 3)
```

Output :

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Indexing in 2D

- Row & column are accessed like [row, col].

```
print(arr2d[0, 0]) # 1 (first row, first column)
```

```
print(arr2d[1, 2]) # 6 (second row, third column)
```

```
print(arr2d[-1, -1]) # 9 (last element)
```

Slicing in 2D

- Works row-wise and column-wise.

```
print(arr2d[0:2, 1:3])
# [[2 3]
#   [5 6]]

print(arr2d[:, 0])
# [1 4 7] (all rows, first column)

print(arr2d[1, :])
# [4 5 6] (second row, all columns)
```

3D Arrays (Tensor)

This is just stacking 2D arrays together.
Shape becomes (depth, rows, cols).

```
arr3d = np.array([ [1, 2], [3, 4]],  
                 [[5, 6], [7, 8]],  
                 [[9, 10], [11, 12]] )  
  
print("3D Array:\n", arr3d)  
print("Shape:", arr3d.shape) # (3, 2, 2)
```

Structure:

- Depth = 3
- Rows = 2
- Cols = 2

```
[  
  [[ 1  2]  
   [ 3  4]],  
  
  [[ 5  6]  
   [ 7  8]],  
  
  [[ 9 10]  
   [11 12]]  
]
```

Indexing in 3D

```
print(arr3d[0, 1, 1]) # 4 (0th depth, 1st row, 1st col)  
print(arr3d[2, 0, 1]) # 10
```

Slicing in 3D

```
print(arr3d[:, 0, :])  
# Take all depths, row=0, all columns  
# [[ 1  2]  
# [ 5  6]  
# [ 9 10]]  
  
print(arr3d[1:, :, :])  
# Take depth 1 onwards (second & third matrices)
```

#ARRAYS

1. Reverse Array: Write a function to reverse an array.

```
import numpy as np

def reverse_array(arr):
    return arr[::-1]

arr = np.array([1, 2, 3, 4, 5])
rev = reverse_array(arr)

print("Original:", arr)
print("Reversed:", rev)
```

Output:

```
Original: [1 2 3 4 5]
Reversed: [5 4 3 2 1]
```

How it Works (Explanation)

- `arr[::-1]` is **slicing with step = -1**.
- It tells NumPy: *“Start from the end and move backwards by 1.”*
- Since NumPy arrays are continuous in memory, it doesn’t need to copy element by element — it just adjusts its **strides (step size in memory)**.
- So reversing is **O(1) view creation** (super efficient), not an O(n) operation like in lists.

If you want an actual **new copy** (not a view), you can do:

```
rev = arr[::-1].copy()
```

2. Find Maximum: Write a function to find the maximum element in an array.

```
import numpy as np

def find_max(arr):
    return np.max(arr) # or arr.max()

arr = np.array([10, 45, 3, 99, 67])
print("Array:", arr)
print("Maximum:", find_max(arr))
```

Output:

```
Array: [10 45 3 99 67]
Maximum: 99
```

How it Works (Explanation)

- `np.max(arr)` → scans through the array internally (written in **C** → way faster than Python loops).
 - Time Complexity: **$O(n)$** (must check all elements).
 - But NumPy does this in **vectorized**, so it feels instant even for huge arrays.
-

Index of Maximum

Sometimes we also need **where** the max is (not just value).

```
def find_max_index(arr):
    return np.argmax(arr) # index of first maximum

Example:
arr = np.array([10, 45, 3, 99, 67])
print("Index of max:", find_max_index(arr)) # 3
```

So with NumPy:

- `np.max(arr)` → gives max value.
- `np.argmax(arr)` → gives index of max value.

3. Search Element: Write a function to search for an element in an array.

```
import numpy as np

def search_element(arr, target):
    result = np.where(arr == target) # returns tuple of indices
    if result[0].size > 0:
        return result[0] # indices where element is found
    else:
        return -1

arr = np.array([10, 20, 30, 40, 20, 50])

print("Search 20:", search_element(arr, 20))
print("Search 99:", search_element(arr, 99))
```

Output:

```
Search 20: [1 4] # found at index 1 and 4
Search 99: -1    # not found
```

How it Works

- `arr == target` → creates a boolean mask.
Example: `arr == 20` → `[False True False False True False]`.
- `np.where(condition)` → returns indices where condition is True.
- If none found, we return -1.

Efficiency:

- Still **$O(n)$** because it checks all elements.
 - But done in **vectorized C code**, so it's blazing fast compared to Python loops.
-

4. Sort Array: Write a function to sort an array in ascending order.

```
import numpy as np

def sort_array(arr):
    return np.sort(arr) # returns a new sorted array

arr = np.array([5, 3, 8, 1, 2])
sorted_arr = sort_array(arr)

print("Original:", arr)
print("Sorted:", sorted_arr)
```

Output:

```
Original: [5 3 8 1 2]
Sorted: [1 2 3 5 8]
```

How it Works

- `np.sort(arr)` → returns a **new sorted copy** (doesn't change original).
 - Internally, NumPy uses a variant of **Quicksort / Heapsort / Mergesort** depending on dtype & size.
 - Time Complexity: **$O(n \log n)$** .
-

Bonus Tricks

1. Sort in place (no extra memory):

```
arr.sort()
print("In-place sorted:", arr)
```

2. Sort in descending order:

```
np.sort(arr)[::-1]
```

So with NumPy:

- `np.sort(arr)` → new sorted array.
 - `arr.sort()` → in-place sort.
-

Sorting

Introduction to Sorting

- In computer science, **sorting** is the process of arranging the elements in a list or collection in a specific, meaningful order. The most common orders are numerical (e.g., 1, 2, 3, ...) and alphabetical, (e.g., a, b, c, ...).

Why is it important?

- **Efficient Searching:** It's incredibly fast to search in a sorted list (e.g., using Binary Search).
 - **Data Presentation:** Users often need to see data ordered by date, name, price, etc.
 - **Easier Analysis:** Finding medians, percentiles, and duplicates is simpler with sorted data.
 - **Adaptability:** An adaptive algorithm's performance becomes better (or more efficient) if the input list is already partially sorted.
-

Classification of Sorting Algorithms

A. Simple Sorts ($O(n^2)$)

- **Bubble Sort**
- **Selection Sort**
- **Insertion Sort**

Best for small datasets or teaching basics.

B. Efficient Sorts ($O(n \log n)$)

- **Merge Sort** (Divide & Conquer, stable, $O(n)$ extra space)
- **Quick Sort** (Divide & Conquer, in-place, but unstable)

Used in large datasets, practical real-world applications.

Python's Built-in Sorting

- `list.sort()` → In-place, modifies original.
- `sorted(iterable)` → Returns new sorted list.

Note :- Both use **TimSort** (stable, adaptive, efficient).

Bubble Sort

Concept: Bubble Sort is a simple comparison-based algorithm. It works by repeatedly stepping through the list, comparing adjacent elements, and swapping them if they are in the wrong order. This process is repeated until the list is sorted.

The pass through the list is repeated until no swaps are needed, which indicates the list is sorted. The larger values "bubble" up to the end of the list with each pass, much like air bubbles rising in water—hence the name.

Bubble Sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

- After each pass, the largest element “bubbles up” to its correct position.
 - We repeat until the list is fully sorted.
-

Flow of Bubble Sort

Step-by-step:

1. Start from the first element.
 2. Compare the current element with the next one.
 3. If $\text{current} > \text{next} \rightarrow$ swap them.
 4. Move to the next pair and repeat until the last unsorted element.
 5. After one pass $\rightarrow$ the largest element is placed at the end.
 6. Repeat the process for the remaining unsorted part of the list.
 7. Stop when no swaps are needed (already sorted).
-

Example Dry Run

Input: [5, 3, 8, 4, 2]

- **Pass 1:** [3, 5, 4, 2, 8]
 - **Pass 2:** [3, 4, 2, 5, 8]
 - **Pass 3:** [3, 2, 4, 5, 8]
 - **Pass 4:** [2, 3, 4, 5, 8]
-

Python Code for Bubble Sort

Here is the Python implementation that follows the logic of the optimized flowchart:

python

```
def bubble_sort(arr):
    n = len(arr)
    swapped = False

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
        else:
            swapped = False
    return arr
```

Example Usage:-

```
my_list = [64, 34, 25, 12, 22, 11, 90]
print("Original list:", my_list)

sorted_list = bubble_sort(my_list.copy())
print("Sorted list: ", sorted_list)
```

Key Points in the Code:

- $n - i - 1$: The inner loop runs one shorter each time because the largest unsorted element "bubbles" to its correct position at the end on each pass.
- **swapped Flag**: This is a crucial optimization. If the list is already sorted or becomes sorted early, the algorithm will break out of the loop, saving unnecessary passes.

Analysis of Bubble Sort

- **Time Complexity:**
 - **Worst-case & Average-case & Best-case : $O(n^2)$** . This happens when the list is in reverse order, requiring every possible comparison and swap.
- **Space Complexity: $O(1)$** . It only requires a constant amount of additional memory space (for variables like n , i , j , `swapped`), making it an **in-place** sorting algorithm.
- **Stability: Yes, Bubble Sort is stable**. It only swaps adjacent elements if they are unequal, so equal elements never swap past each other, preserving their original relative order.

Bubble Sort is a great teaching tool for understanding sorting concepts, but its $O(n^2)$ complexity makes it **highly inefficient** for large datasets.

Selection Sort

The algorithm repeatedly finds the minimum (or maximum) element from the unsorted sublist and swaps it with the leftmost unsorted element, moving the boundary of the sorted sublist one element to the right.

Concept: Selection Sort is another simple comparison-based algorithm. It works by **repeatedly selecting the smallest (or largest) element** from the unsorted part of the list and swapping it with the first unsorted element.

It is called "Selection" Sort because it *selects* the smallest element each time.

- After each pass, the boundary of the “sorted” and “unsorted” parts shifts by one.
 - It’s like picking the best player for your team one by one until the whole team is set. 🏀
-

Step-by-step Process

1. Start with the full array.
 2. Assume the first element is the smallest.
 3. Traverse the rest of the array → find the actual smallest element.
 4. Swap it with the first unsorted element.
 5. Move the “sorted boundary” forward by 1.
 6. Repeat until the whole list is sorted.
-

Example Dry Run

Input: [64, 25, 12, 22, 11]

- **Pass 1:** Min = 11 → Swap with 64 → [11, 25, 12, 22, 64]
 - **Pass 2:** Min = 12 → Swap with 25 → [11, 12, 25, 22, 64]
 - **Pass 3:** Min = 22 → Swap with 25 → [11, 12, 22, 25, 64]
 - **Pass 4:** Min = 25 → Already in place → [11, 12, 22, 25, 64]
-

Python Code for Selection Sort

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i
        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr
```

Example Usage:-

```
my_list = [64, 25, 12, 22, 11]
print("Original list:", my_list)

sorted_list = selection_sort(my_list.copy())
print("Sorted list: ", sorted_list)
```

Step-by-step breakdown using the example [64, 25, 12, 22, 11]:

- **Pass 1 (i=0):** The unsorted list is [64, 25, 12, 22, 11]. The smallest element is 11. Swap 64 and 11.
 - List becomes: [11, 25, 12, 22, 64]
- **Pass 2 (i=1):** The unsorted list is [25, 12, 22, 64]. The smallest element is 12. Swap 25 and 12.
 - List becomes: [11, 12, 25, 22, 64]
- **Pass 3 (i=2):** The unsorted list is [25, 22, 64]. The smallest element is 22. Swap 25 and 22.
 - List becomes: [11, 12, 22, 25, 64]
- **Pass 4 (i=3):** The unsorted list is [25, 64]. The smallest element is 25. It is already in place, so no swap needed.
- The list is now sorted: [11, 12, 22, 25, 64].

Analysis of Selection Sort

- **Time Complexity: $O(n^2)$** for all cases (worst, average, and best).
 - Why? The inner loop always runs $O(n)$ times to find the minimum for each element. This happens even if the list is already sorted. There is no early termination condition.
- **Space Complexity: $O(1)$.** It only requires a constant amount of additional memory space (for variables like n, i, j, min_index), making it an **in-place** sorting algorithm.

- **Stability: No, Selection Sort is not stable.**
 - **Why?** The swapping of elements can change the relative order of equal values.
 - **Example:** Imagine sorting $[4_1, 2, 3, 4_2, 1]$ by value (the subscript distinguishes the two 4s). The algorithm will find the minimum 1 and swap it with the first element 4_1 . This puts 4_1 at the end: $[1, 2, 3, 4_2, 4_1]$. The original relative order of the two 4s (4_1 before 4_2) has been reversed.

Selection Sort is simple and performs well on very small lists or when the cost of swapping is high (as it only does a maximum of n swaps). However, its $O(n^2)$ time complexity makes it inefficient for larger lists.

Insertion Sort

Insertion Sort builds the sorted list one element at a time by taking the next element and inserting it into its correct position among the already sorted elements.

Concept: Insertion Sort is a simple, intuitive sorting algorithm that works similarly to how you might sort a hand of playing cards. It builds the sorted list one element at a time by **inserting** each new element into its correct position within the already-sorted part of the list.

It maintains a sorted sublist (usually at the beginning of the list) and repeatedly takes the next element from the unsorted part and inserts it into the correct spot in the sorted sublist, growing the sorted sublist by one element each time.

- Works just like arranging playing cards in your hand 🃏 : you pick one card and slide it into the right place.

Efficient for small arrays or when the array is **almost sorted**.

Step-by-step Process

1. Assume the first element is already sorted.
 2. Pick the next element (key).
 3. Compare the key with the elements in the sorted part.
 4. Shift larger elements to the right.
 5. Insert the key into its correct position.
 6. Repeat until the entire array is sorted.
-

Example Dry Run

Input: [12, 11, 13, 5, 6]

- **Pass 1 (key=11):** [11, 12, 13, 5, 6]
 - **Pass 2 (key=13):** [11, 12, 13, 5, 6] (no shift)
 - **Pass 3 (key=5):** [5, 11, 12, 13, 6]
 - **Pass 4 (key=6):** [5, 6, 11, 12, 13]
-

Python Code for Insertion Sort

```
def insertion_sort(l):
    for i in range(1,len(l)):
        key = i

        while key > 0 and l[key] < l[key-1]:
            l[key],l[key-1] = l[key-1],l[key]
            key-=1
    return l
```

Example Usage:-

```
my_list = [12, 11, 13, 5, 6]
print("Original list:", my_list)

sorted_list = insertion_sort(my_list.copy())
print("Sorted list: ", sorted_list)
```

Step-by-step breakdown using the example [12, 11, 13, 5, 6]:

- **Pass 1 (i=1):** key = 11, j=0
 - Compare 11 < 12? Yes. Shift 12 right: [12, 12, 13, 5, 6]. j becomes -1.
 - Insert key at j+1 (index 0): [11, 12, 13, 5, 6]
- **Pass 2 (i=2):** key = 13, j=1
 - Compare 13 < 12? No. Exit inner loop.
 - Insert key at j+1 (index 2): [11, 12, 13, 5, 6] (no change)
- **Pass 3 (i=3):** key = 5, j=2
 - Compare 5 < 13? Yes. Shift 13 right: [11, 12, 13, 13, 6], j=1.
 - Compare 5 < 12? Yes. Shift 12 right: [11, 12, 12, 13, 6], j=0.
 - Compare 5 < 11? Yes. Shift 11 right: [11, 11, 12, 13, 6], j=-1.
 - Insert key at j+1 (index 0): [5, 11, 12, 13, 6]
- **Pass 4 (i=4):** key = 6, j=3
 - Compare 6 < 13? Yes. Shift 13 right: [5, 11, 12, 13, 13], j=2.
 - Compare 6 < 12? Yes. Shift 12 right: [5, 11, 12, 12, 13], j=1.
 - Compare 6 < 11? Yes. Shift 11 right: [5, 11, 11, 12, 13], j=0.
 - Compare 6 < 5? No. Exit inner loop.
 - Insert key at j+1 (index 1): [5, 6, 11, 12, 13]
- The list is now sorted.

Analysis of Insertion Sort

- **Time Complexity:**
 - **Worst-case & Average-case: $O(n^2)$.** This happens when the list is in reverse order. Each new element must be compared with all elements in the sorted sublist.
 - **Best-case: $O(n)$.** This occurs when the list is already sorted. The inner loop simply compares the key to one element and stops, making it very efficient for nearly-sorted data. This makes Insertion Sort **adaptive**.
- **Space Complexity: $O(1)$.** It only requires a constant amount of additional memory space (for variables i , j , key), making it an **in-place** sorting algorithm.

Insertion Sort is stable.

- **Why?** It only shifts elements that are *greater* than the key. If it encounters an element *equal* to the key, it stops shifting and inserts the key after it. This preserves the relative order of equal elements.

Insertion Sort is simple, efficient for small datasets or data that is already mostly sorted, and is often used as the final finishing step in more advanced algorithms like Hybrid Sorts (e.g., Timsort).

Merge Sort

Merge Sort is a "**divide and conquer**" algorithm. It works by recursively breaking down a list into smaller sublists until each sublist contains only one element (which is, by definition, sorted). Then, it repeatedly merges these sublists back together in a sorted order until one sorted list remains.

Merge Sort is a Divide & Conquer algorithm:

1. **Divide** → Split the array into two halves.
2. **Conquer** → Recursively sort each half.
3. **Combine** → Merge the two sorted halves into a single sorted array.

The heart of the algorithm is the merge function, which combines two already-sorted lists into one new sorted list.

Step-by-step Process

1. If array has 1 or 0 elements → already sorted.
 2. Find the middle index → split into left and right halves.
 3. Recursively apply merge sort on left half.
 4. Recursively apply merge sort on right half.
 5. Merge both halves into a sorted array.
-

Example Dry Run

Input: [38, 27, 43, 3, 9, 82, 10]

- Split → [38, 27, 43] and [3, 9, 82, 10]
- Further split → [38] [27, 43] and [3, 9] [82, 10]
- Sort + merge step by step →
 - [27, 43]
 - [3, 9]
 - [10, 82]
 - [27, 38, 43]
 - [3, 9, 10, 82]
 - Final merge → [3, 9, 10, 27, 38, 43, 82] 

Python Code for Merge Sort

```
def merge_sort(arr,left,right):
    if len(arr)>1:
        mid = len(arr)//2
        left_arr = arr[:mid]
        right_arr = arr[mid:]

        merge_sort(left_arr,left,mid)
        merge_sort(right_arr,mid+1,right)
        i = 0
        j = 0
        k = 0
        while i<len(left_arr) and j<len(right_arr):
            if left_arr[i]<right_arr[j]:
                arr[k]=left_arr[i]
                i+=1

            elif left_arr[i] > right_arr[j]:
                arr[k] = right_arr[j]
                j+=1
            k+=1

        while i < len(left_arr):
            arr[k]=left_arr[i]
            i+=1
            k+=1
        while j < len(right_arr):
            arr[k]=right_arr[j]
            j+=1
            k+=1

arr=[11,32,44,12,55]
```

Example Usage:-

```
my_list = [38, 27, 43, 3, 9, 82, 10]
print("Original list:", my_list)

sorted_list = merge_sort(my_list) # Returns a new sorted list
print("Sorted list: ", sorted_list)
```

Step-by-step breakdown using the example [38, 27, 43, 3]:

1. **First Split:** [38, 27, 43, 3] -> left = [38, 27], right = [43, 3]
2. **Recurse on Left ([38, 27]):**
 - Split: left = [38], right = [27] -> Base cases reached, return [38] and [27].

- Merge [38] and [27]: Compare 38 and 27 -> [27, 38].
 - 3. **Recurse on Right ([43, 3]):**
 - Split: left = [43], right = [3] -> Base cases reached, return [43] and [3].
 - Merge [43] and [3]: Compare 43 and 3 -> [3, 43].
 - 4. **Final Merge:** Merge the results left_sorted = [27, 38] and right_sorted = [3, 43].
 - Compare 27 and 3 -> add 3. result = [3]
 - Compare 27 and 43 -> add 27. result = [3, 27]
 - Compare 38 and 43 -> add 38. result = [3, 27, 38]
 - Append the remainder of the right list (43). result = [3, 27, 38, 43]
-

Analysis of Merge Sort

- **Time Complexity: $O(n \log n)$** for all cases (worst, average, and best).
 - **Why?** The list is divided in half ($\log n$ times). The merge operation for each level of division takes $O(n)$ time. So, total time is $O(n \log n)$.
- **Space Complexity: $O(n)$.**
 - **Why?** The merge function requires a temporary auxiliary array to combine the lists. This is the algorithm's main drawback; it is **not an in-place** sort.
- **Stability: Yes, Merge Sort is stable.**
 - **Why?** The merge function uses $\leq$ when comparing elements. This means if two elements are equal, the one from the *left* sublist is taken first, preserving its original order relative to the one in the right sublist.

Merge Sort is a classic, efficient, and predictable algorithm. Its $O(n \log n)$ performance guarantee makes it suitable for sorting large lists, especially when stability is required. It's also excellent for sorting linked lists and is used in external sorting (sorting data too large for RAM).

Quick Sort

Concept: Like Merge Sort, Quick Sort is a "**divide and conquer**" algorithm. However, its strategy is fundamentally different. It works by selecting a '**pivot**' element from the list and **partitioning** the other elements into two sublists:

1. Elements *less than* the pivot.
2. Elements *greater than* the pivot.

The pivot is then in its final sorted position. The algorithm then recursively applies the same process to the left and right sublists.

The key to the algorithm's efficiency is the partition function, which places the pivot in its correct position.

Step-by-step Process

1. Choose a pivot (can be first, last, random, or median element).
2. Rearrange (partition) array so that:
 - All elements < pivot are before it.
 - All elements > pivot are after it.
3. Recursively apply Quick Sort on left and right subarrays.
4. Stop when subarray size ≤ 1 .

Example Dry Run

Input: [10, 7, 8, 9, 1, 5], pivot = 5

- Partition: [1] [5] [10, 7, 8, 9]
 - Left $\rightarrow$ [1] (already sorted)
 - Right $\rightarrow$ pivot = 9 $\rightarrow$ [7, 8] [9] [10]
 - Sorted result $\rightarrow$ [1, 5, 7, 8, 9, 10]
-

Python Code for Quick Sort

```
def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1

    if low < high:
        pi = partition_lomuto(arr, low, high)

        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def partition_lomuto(arr, low, high):
    pivot = arr[high]

    i = low - 1

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]

    return i + 1
```

Example Usage :-

```
my_list = [10, 80, 30, 90, 40, 50, 70]
print("Original list:", my_list)

quick_sort(my_list) # Sorts the list in-place
print("Sorted list: ", my_list)
```

Step-by-step breakdown of one partition on [10, 80, 30, 90, 40, 50, 70] (pivot=70):

- i starts at -1, j starts at 0.
- j=0: 10 <= 70 -> i=0, swap arr[0] with arr[0] (no change). List: [10, 80, 30, 90, 40, 50, 70]
- j=1: 80 <= 70? **No**. Do nothing.
- j=2: 30 <= 70 -> i=1, swap arr[1] (80) with arr[2] (30). List: [10, 30, 80, 90, 40, 50, 70]
- j=3: 90 <= 70? **No**. Do nothing.
- j=4: 40 <= 70 -> i=2, swap arr[2] (80) with arr[4] (40). List: [10, 30, 40, 90, 80, 50, 70]
- j=5: 50 <= 70 -> i=3, swap arr[3] (90) with arr[5] (50). List: [10, 30, 40, 50, 80, 90, 70]
- Loop ends. Now place pivot: swap arr[i+1] (arr[4] = 80) with arr[high] (70). Final List: [10, 30, 40, 50, 70, 90, 80]

- Return pivot index $pi = 4$. All elements left of index 4 are ≤ 70 , all elements right are > 70 .
-

Analysis of Quick Sort

- **Time Complexity:**
 - **Best-case & Average-case: $O(n \log n)$.** This happens when the partition always picks a median element, splitting the list into roughly equal halves.
 - **Worst-case: $O(n^2)$.** This occurs when the partition always picks the smallest or largest element as the pivot (e.g., sorting an already sorted list with the first/last element as pivot). The recursion becomes deeply unbalanced.
- **Space Complexity: $O(\log n)$** on average for the recursion call stack. In the worst case, it can be $O(n)$. It is an **in-place** sort, meaning it uses minimal extra memory beyond the recursion stack.
- **Stability: Quick Sort is not stable.** The partitioning step can swap elements and change the relative order of equal values. For example, an equal element chosen as the pivot might be swapped past other equal elements.
- **Optimizations:** Real-world implementations (like Python's sorted) use optimizations like:
 - **Randomized Pivot:** Choosing a random pivot to avoid the worst-case scenario on sorted data.
 - **Median-of-Three:** Choosing the median of the first, middle, and last elements as the pivot.
 - **Insertion Sort for Small Sublists:** Switching to Insertion Sort for small sublists (e.g., size < 10) to avoid recursion overhead.

Quick Sort is very efficient in practice and is often the preferred choice for sorting in programming languages and libraries due to its excellent average-case

Problem 1 : Given an array of integers `nums` and an integer `target`, return *indices* of the two numbers such that they add up to `target`.

- Input: `nums = [2,7,11,15]`, `target = 9`
- Output: `[0,1]`

Problem 2 : An array contains `n` distinct numbers from `0..n`, but one number is missing. Find it.

- Input: `[3,0,1]`
- Output: `2`

Problem 3 : Given an array `nums`, find the element that appears more than $n/2$ times.

- Input: `[2,2,1,1,1,2,2]`
- Output: `2`

Problem: Given an integer array `nums` and integer `k`, return the `k`-th largest element.

- Input: `[3,2,1,5,6,4]`, `k=2`
- Output: `5`